



Protocol Audit Report

Version 1.0

web3pavlou

March 30, 2026

Protocol Audit Report

web3pavlou

March 28, 2026

Prepared by: web3pavlou Lead Security Researcher: web3pavlou

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
 - Scope
 - Roles
- Executive Summary
 - Issues found
- Findings
- High
- Medium
- Low
- Informational
- Gas

Protocol Summary

PasswordStore is a protocol dedicated to store and retrieves a user's password. It is designed to be used by a single user and not by multiple. Only the owner should be able to access this password.

Disclaimer

The web3pavlou team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

The findings described in this document correspond to the following commit hash:

```
1 cd8d6a858640776c046a37452fd2d795a3ec56d6
```

Scope

```
1 ./src/  
2 #-- PasswordStore.sol
```

Roles

Owner: The user who can set the password and read the password. Outsiders: No one else should be able to set or read the password.

Executive Summary

N/A

Issues found

Severity	Number of issues found
High	2
Medium	0
Low	0
Info	1
Total	3

Findings

High

Description: All data stored on-chain is visible to anyone and can be read directly from the blockchain. The `PasswordStore::s_password` variable is intended to be a private variable and only be accessed through `PasswordStore::getPassword` function, which is intended to be called only from the owner of the contract.

We show one such method of reading any data off chain below.

Impact: Anyone can read the private password, severely breaking the functionality of the protocol.

Proof of Concept: The below test cases show how anyone can read the password directly from the blockchain. 1. Create a locally running chain `bash make anvil` 2. Deploy the contract to the chain `bash make deploy` 3. Run the storage tool We use 1 because that's the storage slot of `s_password` in the contract.

```
1 cast storage <CONTRACT_ADDRESS_HERE> 1 --rpc-url https://127.0.0.1:8545
```

You'll get an output that looks like this: `0x6d7950617373776f726400000000000000000000000000000000000000000000`

You can then parse that hex to string with:

[illegible]

And get an output like this:

```
1 myPassword
```

Recommended Mitigation: The overall architecture of this contract should be rethought. One could encrypt the password off-chain, and then store the encrypted password on-chain. This would require the user to remember another password off-chain to decrypt the password. You 'd also probably likely want to remove the view function as you wouldn't want the user to accidentally send a transaction with the password that decrypts his password.

[H-2] PassWordStore::setPassword has no access control, meaning a non-owner could change the password.

Description: The `PasswordStore : setPassword` is set to be an `external` function , however , the natspec of the function and overall purpose of the smart contract is that `This function allows only the owner to set a new password.`

```
1 function setPassword(string memory newPassword) external {
2   @> // @audit - Missing access control
3     s_password = newPassword;
4     emit SetNetPassword();
5 }
```

Impact: Anyone can set/change the password of the contract severely breaking the contract's intended behavior.

Proof of Concept: Add the following to the `PasswordStore::PasswordStore.t.sol` test file.

Code

```
1 function test_anyone_can_set_password(address randomAddress) public
2 {
3     vm.assume(randomAddress != owner);
4     vm.prank(randomAddress);
5     string memory expectedPassword = "myNewPassword";
6     passwordStore.setPassword(expectedPassword);
7
8     vm.prank(owner);
9     string memory actualPassword = passwordStore.getPassword();
10    assertEq(expectedPassword, actualPassword);
11 }
```

Recommended Mitigation: Add an access control conditional to the `setPassword` that only the owner of the password can set the new password and an error declaration at the top.

```
1 error PasswordStore__NotOwner();
2 .....//rest of the code
3 if(msg.sender != s_owner){
4     revert PasswordStore__NotOwner();
5 }
```

Informational

[I-1] Incorrect NatSpec for PasswordStore::getPassword

Description:

```
1 /*
2  * @notice This allows only the owner to retrieve the password.
3  * @param newPassword The new password to set.
4  */
```

The `PasswordStore::getPassword` function has the signature `getPassword()`, but the NatSpec comment incorrectly includes a parameter (`newPassword`) that does not exist. This implies a function signature of `getPassword(string)`, which is inaccurate.

Impact:

The NatSpec documentation is misleading and does not accurately reflect the function's interface.

Recommended Mitigation:

Remove the incorrect `@param` line from the NatSpec comment.

```
1 - * @param newPassword The new password to set.
```